UTN AVELLANEDA
Tecnicatura Universitaria en Programación

Materia:	Programación II		
Nivel:	2º Cuatrimestre		
Tipo de Examen:	Primer Parcial		
Apellido(1):		Fecha:	23 oct 2026
Nombre/s(1):		Docente a cargo(2):	Baus / Rocha
División(1):	122-2	Nota(2):	
DNI(1):		Firma(2):	

(1) Campos a completar solo por el estudiante en caso de imprimir este enunciado en papel.
(2) Campos a completar solo por el docente en caso de imprimir este enunciado en papel.

Sistema de Gestión de Aventureros de un Reino Versión 1.1

El Reino de Eldoria cuenta con un gremio encargado de registrar y administrar los aventureros que participan en expediciones y misiones.

Existen tres tipos de aventureros: guerreros, magos y arqueros.

Todos los aventureros poseen un código de identificación, un nombre y un rango, que puede ser BRONCE, PLATA u ORO.

Cada tipo de aventurero tiene una característica particular:

- Los guerreros poseen un valor de fuerza.
- Los magos poseen una cantidad de puntos de maná.
- Los arqueros poseen un valor de precisión.

Todos los aventureros pueden realizar un ataque y poseen un poder de combate que se calcula de manera diferente según su tipo.

Además, los guerreros y los arqueros pueden participar en entrenamientos especiales.

Dos aventureros se consideran iguales cuando tienen el mismo código y el mismo nombre, independientemente de su tipo, rango y características particulares.

Reglas de negocio

El poder de combate se calcula mediante las siguientes reglas:

Tipo	Cálculo del poder
Guerrero	fuerza × 2
Mago	maná + 10
Arquero	precisión × 3

Los valores de fuerza, maná y precisión deben ser mayores que cero. El código y el nombre no pueden ser nulos ni estar en blanco, y el rango es obligatorio.

Funcionalidades requeridas

El gremio debe permitir:

1. `agregarAventurero()`: incorporar cualquier tipo de aventurero. Si ya existe uno considerado igual, se debe lanzar una excepción personalizada.
2. `obtenerAventureros()`: devolver todos los aventureros registrados.
3. `filtrarPorRango()`: devolver los aventureros de un rango determinado.
4. `obtenerAventurerosPoderosos()`: devolver los aventureros cuyo poder de combate sea mayor o igual al valor recibido.

El sistema también debe permitir mostrar los ataques, los entrenamientos disponibles y el poder de combate de los aventureros mediante el programa principal proporcionado.

Se solicita

1. Realizar el diagrama de clases completo utilizando Umletino.
2. Implementar en Java las clases, interfaces, enumerados y excepciones necesarios.
3. Resolver todas las funcionalidades solicitadas.
4. Completar los métodos auxiliares del programa principal entregado.
5. Documentar con Javadoc las clases y los métodos principales.

Criterios de corrección:

1. **Aprobación**:
   - El diagrama de clases debe reflejar correctamente la jerarquía de herencia.
   - Las clases y atributos deben estar correctamente implementados.
   - Los métodos solicitados deben funcionar adecuadamente.
   - El manejo básico de excepciones debe estar presente, especialmente al agregar duplicados.
2. **Promoción**:
   - El diagrama debe incluir detalles como visibilidad de atributos y métodos, así como relaciones adicionales (por ejemplo, composición).

- El código debe seguir principios de POO, con buen manejo de encapsulamiento y visibilidad de atributos.
- Se espera el uso de enum e interface.
- El manejo de excepciones debe ser avanzado, con excepciones personalizadas y mensajes claros.
- Se valora el uso de colecciones eficientes.
- Buena modularización del código.

Programa de Prueba

```
public static void main(String[] args) {
        GremioAventureros gremio = new GremioAventureros("Guardianes de Eldoria");

        gremio.agregarAventurero(new Guerrero("G-01", "Arin", Rango.ORO, 30));
        gremio.agregarAventurero(new Mago("M-02", "Lyra", Rango.PLATA, 45));
        gremio.agregarAventurero(new Arquero("A-03", "Nox", Rango.BRONCE, 18));
        gremio.agregarAventurero(new Guerrero("G-04", "Borin", Rango.PLATA, 20));
        gremio.agregarAventurero(new Mago("G-01", "Arin", Rango.BRONCE, 99));

        System.out.println("Aventureros registrados:");
        mostrarAventureros(gremio.obtenerAventureros());
        System.out.println("\nAtaques:");
        mostrarAtaques(gremio.obtenerAventureros());
        System.out.println("\nEntrenamientos:");
        mostrarEntrenamientos(gremio.obtenerAventureros());
        System.out.println("\nPoder de combate:");
        mostrarPoderes(gremio.obtenerAventureros());
        System.out.println("\nAventureros de rango PLATA:");
        mostrarAventureros(gremio.filtrarPorRango(Rango.PLATA));
        System.out.println("\nAventureros con poder >= 55:");
        mostrarAventureros(gremio.obtenerAventurerosPoderosos(55));

    }
    private static void mostrarAventureros(List<Aventurero> aventureros) {

    }

    private static void mostrarAtaques(List<Aventurero> aventureros) {

    }

    private static void mostrarEntrenamientos(List<Aventurero> aventureros) {

    }

    private static void mostrarPoderes(List<Aventurero> aventureros) {

    }
}
```